# Swarma: платформа для роёв автономных агентов

Рантайм на машинах владельца, протокол совместной разработки и цепочка идентичностей на хэш-криптографии

Алексей Григорьев, Иван Опарин
Relux Works
github.com/relux-works

Октябрь 2026, пятая редакция, версия 5.2

**Аннотация.** Swarma — платформа для роёв автономных агентов из трёх слоёв: рантайма на машинах одного владельца, протокола совместной разработки для независимых владельцев и цепочки идентичностей. Общая задача слоёв — отделить способность модели предложить действие от полномочия его совершить. Swarma Runtime задаёт границы пользователей ОС с cgroup на поколение учётки, хранение ключей вне агента, делегирование прав и управляемую доставку в сессии. Swarma Protocol связывает работу над единой авторитетной средой с журналом, кворумом, независимым ревью и атомарным продвижением точно проверенного кандидата. Swarma Chain даёт общее состояние личностей, имён, денег и аттестаций. Её консенсус опирается только на хэш-функции, а майнинг — собственный, без объединённого. В этой редакции дизайн финальный: решения приняты, а нерешённое перечислено отдельно. Для ограниченного фрагмента интеграции воспроизведён исчерпывающий перебор: 1090 состояний без нарушений, а отключение каждого из четырёх правил даёт контрпример. Это не доказательство всей платформы. Полезность роя, качество ревью, развёртывание рантайма и переносимая репутация требуют отдельных проверок. Вклад работы — композиция границ доверия с явными допущениями и план экспериментов, способных опровергнуть её пользу.

## 1. Введение

Агент разработки всё реже ограничивается одним запросом: он читает чужие документы, запускает инструменты и меняет реальные системы. Среда при этом часто знает только запустившего процесс пользователя, и его права достаются всем агентам. Инъекция в промпт использует эти права руками самого агента. Просьба не читать секрет слабее границы, за которой процесс прочитать его не может.

Переход от одного агента к рою добавляет другую проблему: как объединить ёмкость независимых владельцев на общем проекте. Среда разработки бывает дорогой или невоспроизводимой: особый тулчейн, лицензии, оборудование, данные и ключи подписи привязаны к машине. Параллельные агенты одного владельца расходуют одну квоту провайдера. Чужой владелец не должен передавать проекту свой логин, а проект не должен отдавать ему административный доступ.

Крупные координируемые рои доступны прежде всего вендорам моделей и организациям с собственными ресурсами; открытые проекты по-прежнему координируются через людей. Выигрыш от автономной координации — гипотеза. Проверять её нужно на одинаковых внешних задачах и с учётом цены координации.

В формальной математике результат агента проверяет ядро доказательств. У обычного программного проекта такого оракула принятия нет: успешная сборка не доказывает соответствия намерению. Поэтому принятие конструируется из контракта задачи, проверок, независимого суждения и правил интеграции. Например, исправление может пройти тесты на старой базе, но стать неверным после изменения соседнего интерфейса.

Пятая редакция фиксирует финальный дизайн всех трёх слоёв. Мы описываем, где возникает авторитет, кто вправе менять состояние и к каким версиям относятся свидетельства. Если эта работа расходится со спецификацией модуля, права спецификация. Где компонент уже работает, это сказано явно. Решение не означает реализации, а проверка фрагмента не переносится на всю систему. Нерешённое собрано в заключении.

## 2. Модель: три слоя и участники

**Swarma Runtime, слой 2,** организует агентов на машинах одного владельца: учётки ОС, сеть, ключи, гранты, сессии и процессы. Отдельные компоненты уже работают, а граница отдельного пользователя ОС для каждого автономного агента строится. **Swarma Protocol, слой 1,** описывает совместную разработку независимых владельцев над единой средой исполнения: кор-ноды ведут журнал, рабочие ноды предлагают изменения, а принятие проходит независимое ревью. **Swarma Chain, слой 0,** — общее состояние денег, имён, личностей и социальных записей. Рантайм с первого дня строится в расчёте на цепочку, а цепочка делается параллельно, с приоритетом у рантайма. До её запуска личности ведёт реестр владельца с теми же идентификаторами.

Владелец отвечает за свои машины, агентов и источник ёмкости модели. Оператор совершает привилегированные действия; это роль человека, а не ещё один агент с широким промптом. Агент — длительно живущий участник с личностью и ограниченными полномочиями. Контрибьютор отдаёт проекту ёмкость своего агентского цикла; донор — владелец такой ёмкости. Проект задаёт цель, среду и правила принятия. Совмещение ролей дополнительных прав не даёт.

Граница рантайма рассчитана на агента, которого может захватить инъекция. Ядро ОС, root, установка платформы и доверенные посредники входят в основание доверия. Граница протокола рассчитана на недоверенные рабочие ноды и, на соответствующем уровне, на неисправные кор-ноды; среда проекта доверена в отношении целостности. Swarma Chain рассчитана на открытую сеть, где подпись подтверждает происхождение записи, но не честность участника. Модели противника различны: проверка подписи не доказывает соблюдения сетевой политики, а кворум не исправляет ложный отчёт скомпрометированной среды.

У слоёв есть общий сценарий. Внешний агент проверяет кандидата проекта, не получая его секретов, а проект проверяет полномочия и происхождение вердикта. Аттестация исхода в цепочке позволяет агенту предъявить историю другому проекту. Решение о допуске при этом остаётся у каждого проекта.

## 3. Сквозные принципы

**Ключ задаёт криптографическую личность, но не лежит у агента.** В рантайме агент просит ключника подписать сообщение, а другим владельцам предъявляет ячейку личности. Ячейка хранит текущие ключи, хэш следующих ключей и ссылку на родителя. Компрометация дома агента не раскрывает мастер-секрета. Злоупотребление разрешённой подписью этот принцип не запрещает: полномочия проверяются отдельно.

**Полномочия только сужаются.** Грант — подписанное разрешение на действия над конкретными целями. Делегирование не расширяет действий, области и срока родителя. В протоколе цель содержит полномочие, полномочие — задачу, а задача — ревизию в разрешённой области записи. Например, разрешение менять реализацию функции не даёт продьюсеру права переписать критерии её принятия. Ячейки личностей в цепочке сами по себе грантов в чужом рантайме не выдают.

**Журнал авторитетен, сессия модели — кэш.** Назначение, результат, передача и основание принятия переживают потерю контекста. Преемник восстанавливает обязательства по записям их владельцев, а не по уверенной фразе старой сессии. У доски, хоста сессий и кипера свои области состояния; один глобальный журнал всех событий не нужен.

**Суждение моделей остаётся вне консенсуса.** Валидатор проверяет подписи, версии, вложенность областей и допустимость перехода, но не голосует за красоту кода. Мост может дополнительно сузить допуск модельной проверкой, но модель не создаёт отсутствующего права. В цепочке узел проверяет конверт социальной записи, а смысл детерминированно вычисляют индексаторы.

**Одобрение адресовано точному предмету.** В протоколе адрес аттестации проверки — версия контракта, кандидат, политика проверки и окружение. Другое состояние требует новых оснований принятия. Аттестация в цепочке несёт хэш работы и версии контракта и политики. Слов «это уже проверяли» недостаточно, если проверяли другую базу или другой контракт.

**Непроверяемое не становится разрешённым.** Неизвестный исход не равен успеху, а нечитаемый реестр отзыва — отсутствию отзыва. Недоступный реестр не считается выданным грантом. В рантайме неготовая граница не называется принудительной. В протоколе отсутствие независимого ревьювера ведёт к ожиданию или к отдельно авторизованному изменению условий.

**Решение человека удостоверяется его ключом.** Оператор подписывает точное действие телефоном после показа содержимого и биометрии; запасной путь — отдельная операторская учётка с подтверждением в терминале. Ключ телефона хранится в защищённом чипе и регистрируется QR-сопряжением, а второе устройство обязательно. Агентская сессия присутствия человека не заменяет. Расширяющие действия — выдача гранта, подключение подписки, постоянная команда, продление постоянных грантов — без такой подписи не исполняются.

## 4. Swarma Runtime: рой на машинах одного владельца

**Граница учётки, нагрузки и сети.** Каждому автономному агенту назначается отдельный пользователь ОС с собственным домом и процессами. При корректных правах, закрытой сети и доверенном root другой агент не читает его файлов и па-

мяти и не пишет ему в терминал. Самостоятельный репозиторий нужен и для кода: связанный `git worktree` между учётками оставлял бы общие настройки и хуки. На Linux у каждого поколения учётки своя cgroup v2 под управлением root: `cpu.max`, `memory.max`, `pids.max` и лимиты ввода-вывода. Остановка — мягкий сигнал, затем `cgroup.kill`. Доказательство полной остановки — `populated 0` в `cgroup.events`, а не обход процессов. Это примитивы ядра, без обязательных контейнеров, образов и виртуальных машин. На macOS границу держит квалифицированный профиль Seatbelt, и пока совокупные лимиты не доказаны, инвентарь macOS помечен как частичный.

Учётка сама по себе сеть не ограничивает. Файрвол по UID направляет агента только к слушателю доверенного прокси его профиля, а профиль выбирает политика машины. Unix-сокеты допускают собеседника по данным ядра. Сетевая заморозка идёт в строгом порядке: сужение правила ядра, сброс сессий прокси, снятие слушателя; запись о заморозке переживает перезапуск. CONNECT ограничивает имя, порт и адрес соединения, но не содержимое TLS-туннеля, поэтому разрешённый адресат остаётся каналом выноса. Сегодня работают кооперативные профили прокси, но принуждением они не являются.

Менеджер пользователей (`swarma-user-manager`) создаёт и выводит учётки, а диспетчер (`swarma-dispatcher`) допускает заявки, учитывает квоты и вызывает узкий запускатель. Поколение учётки отличает новое использование UID от старого, поэтому повторный номер не возвращает чужих прав. Допуск каждого действия определяет кортеж: личность работника, поколение учётки, назначение, раунд, дайджест плана и состояние отзыва. План запуска передаётся на дескрипторе 3: на Linux это `memfd` с печатями, на macOS — неименованный файл с правами `0400`. При неизвестном исходе создания или запуска диспетчер сверяет тот же эффект, а не создаёт новый. Вывод учётки после сбоя идёт вперёд, и дом удаляется: сканирование файлов не доказывает отсутствия спрятанного токена. Сохранения дома оператором в первой версии нет.

Службы рантайма — мост, хост сессий, исполнитель рабочего места — пишут почту, заявки, аудит и спулы за работника по общему контракту бюджета. Действуют лимиты байтов и количества на принципала и на узел и явные квитанции обратного давления. Записи остановки и отзыва имеют исключительный резерв.

**Ключник, гранты и сдерживание.** Ключник — отдельный привилегированный сервис с мастер-секретом. Путь личности стабилен: «назначение / продукт / проект / агент», он не зависит от UID и каталога. Ключи каналов рантайма выводятся по SLIP-0010, а ключи цепочки — хэш-функцией SHAKE256 от пути и номера ротации. Собеседника определяют UID от ядра и активное поколение регистрации, а не имя из запроса. Плановая смена мастер-секрета идёт через новый корень, а непрерывность держит ячейка владельца, ключи которой из мастер-секрета не выводятся. При компрометации корень отзывается, и ячейка владельца переводит ячейки агентов на ключи нового корня. Модуля ключника пока нет.

Грант проверяется у получателя действия: подписи всей цепочки, сужение, сроки, условия и отзыв. Оркестратор просит, а не судит. Права ребёнка зависят и от диспетчера, и от вызывающего, и отзыв оркестратора отрезает его потомков. Реестр ключей и грантов — одна служба владельца с журналом в git и подписанными снимками не старше 60 секунд. Со снимком старше новые аренды и продления не выдаются.

Сдерживание больше, чем отзыв гранта. У него три состояния: полномочие броке-

ра отозвано, локальное исполнение и сеть отгорожены, учётные данные провайдера аннулированы. Всё это делает одна операция «отозвать и остановить». При потере доверенного времени удержание длится не дольше 10 минут по монотонным часам, после чего всё, что зависит от времени, останавливается. Брокер учётных данных (`swarma-credential-broker`) ограничивает выдачу токена, но процесс модели способен прочитать выданный ему токен. Немедленный отзыв не отзывает токена, который уже находится в процессе.

**Сообщения и сессии.** Мост принимает подписанные сообщения поверх шифрования носителя; членство в комнате Matrix прав не выдаёт. Провайдер сохраняет событие, ворота проверяют точные байты, подпись, гранты и повтор, и только затем сообщение попадает в долговечную очередь. Агент получает короткий звонок и вытягивает тело из почтового ящика как данные. Проверка небольшой моделью может только сузить допуск. Исходящие байты агент подписывает собственным ключом, и мост их не переписывает. Сегодня работают звонок и вытягивание.

Хост сессий (`swarma-session-host`) отделяет службу учёта от раннера под учёткой агента. У ввода один писатель, а старый контроллер отсекается эпохой. При перезапуске доставка закрыта до сверки старого исполнения, привязок и неизвестных доставок. Запись в канал ещё не доказывает применения сообщения. Привилегированный старт остаётся у диспетчера. После перезагрузки постоянные команды поднимает кипер, а остановленное человеком остаётся остановленным. Общий стоп — одна команда оператора с телефона, возобновление — только его подписью. Сегодня работает демон сессий внутри доски.

**Обязательства и конфигурация.** Swarma Keeper — главный цикл узла и ядро обязательств: кто кому что должен и к какому сроку. Передача завершается первым подтверждённым действием нового держателя, а не ответом «принял». Если новый ревьювер согласился, но не начал работу, обязанность остаётся у прежнего держателя и эскалируется. Пока пробуждение не доказано, каждый шаг эскалации идёт двумя уведомлениями. Журнал подписан и входит в резервную копию. Без проверенного времени работают только монотонные часы и продлений нет; продлений не больше двух на эпизод.

Swarma Curator владеет профилями, скиллами, MCP и содержимым управляемых домов, и второго менеджера профилей нет. Без проверки готовности запуска нет; известное отсутствие логина и неизвестность различаются. Любой MCP подключается только с явного согласия оператора, а писатель конфигурации один — Curator. Неквалифицированные хранилища и клиенты получают отказ. Доверие рабочим областям управляемых домов ведёт журнал доверия (CIP-0020). Для живой сессии используется собственный снимок, потому что управляемый дом может обновляться. Профили, управляемые дома, скиллы и аудит работают сегодня.

**Удалённая работа и публикация.** Удалённому агенту проект выделяет рабочее место под отдельной учёткой. «Модель думает у донора, руки — у нас»: логин провайдера остаётся у донора, а вызовы инструментов исполняются у проекта. Исходящей сети у рабочего места нет, а входящее SSH-соединение — другая граница: принципал привязан к поколению учётки. Порядок закрытия поколения такой: заморозка, закрытие SSH-входа, `cgroup.kill`, экспорт, замена. Проверка терминала при инициализации защищает от случайного запуска, но согласием не является. Связку с учётными данными провайдера донор активирует ключом с присутствием человека — приложением на телефоне или ключом FIDO2. Донор подписывает кортеж ат-

тестации над измеренным экспортом: назначение, раунд, база, дайджест результата, дайджест плана и голова квитанций. Слепой подписи нет. Мост донора получает экспорт целиком и сам пересчитывает дайджест. Назначение, раунд, базу и план он сверяет со своим подписанным заданием, а голову квитанций — с собственной цепочкой ретранслированных вызовов. Агент видит итоговый диф до подписи, а копию подписанного донор хранит у себя.

У донора три варианта установки. В A локальные инструменты убраны из проверенной сборки харнеса, но отдельной границы ОС нет. В B харнес и мост работают под разными UID. В C харнес заперт песочницей ОС; на Linux граница C — приватное сетевое пространство имён без внешнего маршрута. После квалификации B и C вариант A для новых доноров закрывается. Варианты защищают донора, а не проект: проект защищает независимое ревью каждого кандидата, поэтому писать в рабочее место может и A.

Доска задач (`task-board`) связывает продьюсера, ревьювера и Change Request, а принятый кандидат интегрируется через compare-and-set — замену ожидаемой головы. Этот цикл работает сегодня. Публикует результат служба действий с репозиторием под своей системной учёткой по подписанному доской запросу на точного кандидата. Ни агент, ни оркестратор доступа к хостингу кода не имеют.

Маршрутизатор моделей ранжирует только уже допущенные конфигурации по меткам задачи, измерениям качества и бюджету. Он не запускает процессов, не расширяет допуска и не вызывает модель для выбора. Неизвестное качество не разрешает выбрать кандидата. Режим экономии включается только явным решением оператора, а обучаемого маршрутизатора в первой версии нет. Библиотека и командный интерфейс работают.

Память агентов — ограниченная доказательная память: утверждения, ссылки на источники, обоснования, оценки и ревизии. Она отвечает, какие допущенные свидетельства поддерживают или опровергают утверждение, но не что истинно и кому можно действовать. Пустой неполный поиск означает неизвестность. Издатель, субъект и отправитель — ячейки личностей, а основание представительства — грант в реестре владельца. Память состоит из пространств агента, проекта и платформы. В первой рабочей версии это git-репозитории: агенты пишут сами, каждую запись подписывает её отправитель, а каждый читатель проверяет её при чтении. Снимок памяти приходит отдельным блоком данных и в инструкции не попадает. Реализации и измеренного выигрыша пока нет.

Таблица ниже — дорожная карта, а не перечень действующих гарантий.

| Возможность | Состояние |
|---|---|
| Доска и её демон сессий; профили, дома и аудит Swarma Curator; маршрутизатор; кооперативные сетевые профили; звонок и вытягивание | работает сегодня |
| Контракты v0 брокера и setup-токена | опубликованы |
| Менеджер пользователей, диспетчер, ключник, брокер, хост сессий, мост, Swarma Keeper, рабочее место, служба репозитория, память | дизайн; кода нет |

## 5. Swarma Protocol: много владельцев, одна среда

**Топология.** Среда исполнения проекта — единственный логически авторитетный экземпляр основной ветки, санбоксов и проверок. Физически допустимы исполнительные реплики, но голова меняется в одном месте. Среда пассивна: агентские циклы работают на кор-нодах и машинах контрибьюторов, а по сети проходят команды, вывод, хэши и подписи. Кор-ноды ведут журнал и проверяют правила; рабочие ноды не доверены. Единство среды экономит её воспроизведение, но не освобождает от проверки её честности. Распределённого развёртывания протокола пока нет.

**Журнал и кворум.** Запись связана хэшем с предыдущей, а дерево Меркла позволяет доказать включение под подписанным снимком. Тела адресуются по содержимому, их доступность подтверждают независимые реплики. Сертификат кворума, QC, означает достаточное число подписей для объявленного уровня доверия. Лидер, упорядочивающий записи, и оркестратор, планирующий задачи, — разные роли. Среда проверяет QC, включение и актуальность разрешения на границе исполнения и хранит высшую принятую версию журнала, чтобы не принять старый снимок. Принимающий писатель — держатель аренды корневого оркестратора; он ротируется вместе с арендой.

На L0 одна кор-нода: QC — одна подпись, кор и среда доверены. На L1 при $n \geq 2$ нужны $t > n/2$ подписей и честный секвенсер. Это препятствует одиночному несанкционированному решению, но единственность истории зависит от секвенсера. На L2 при $n \geq 3f + 1$ используются BFT-консенсус и порог $q = \lceil (n + f + 1)/2 \rceil$ [1, 2]. Два кворума пересекаются минимум в $2q - n > f$ узлах, то есть хотя бы в одном честном. При $n = 3f + 1$ порог равен $2f + 1$. Фиксированные $2f + 1$ при большем $n$ неверны: при $n = 5$ и $f = 1$ они дали бы порог 3, и два кворума по три подписи могли бы пересечься ровно в одном, злонамеренном, узле. Формула даёт порог 4. Подписи журнала — Ed25519; их переход на хэш-подписи входит в план миграции корней доверия (раздел 7).

**Контракт и полномочия.** Контракт задачи фиксирует родительскую цель, область записи, версии входов, ожидаемый результат, критерии принятия, профиль подтверждаемых свойств, риск и бюджеты. План отвечает на другой вопрос: какие инструменты, гранты и лимиты разрешены исполнению. Его дайджест включён в назначение и сверяется на вызове. Новые критерии создают новую версию контракта с отдельной авторизацией. Изменения в CI, хуках, инструкциях агентов и симлинках одобряет детерминированный предикат. Он вычисляется над текущим контрактом, политикой риска, точным кандидатом и полным множеством помеченных путей накопленной дельты от базы до кандидата, с привилегированным порогом.

Валидаторы проверяют допустимость перехода, вложенность полномочий, текущие версии, независимость ролей, аренду, лимиты и доступность тел, но не качество реализации. Две задачи с пересекающимися областями записи одновременно не исполняются: область удерживается до интеграции или отмены. Непересекающиеся пути не исключают семантического конфликта: новый тип в одном модуле может сломать использование в другом. Поэтому принятие относится к целому кандидату, а не только к локально изменённым файлам.

**Среда и свидетельства.** Назначение открывает доступ только к санбоксу задачи. Копия дерева границей не является: нужны квалифицированные ограничения ОС, то есть пользователь ОС с сетевой границей рантайма. Среда публикует инвентарь того,

что умеет ограничивать, и требование полной границы частичной не удовлетворяется. Команды связаны цепочкой квитанций (receipts) с входным и результирующим состоянием. Квитанция удостоверяет происхождение отчёта среды, но не воспроизводимость: для неё нужны сохранённые входы, версии тулчейна, зависимости и существенные внешние ответы.

**Оркестратор по аренде.** Оркестратор декомпозирует цель и предлагает назначения в пределах своих полномочий. Аренда задаёт область, эпоху и fencing-токен; просроченная запись отвергается и сохраняется как расхождение. Преемник определяется ротацией. Плановая передача требует покоя: нет незавершённых вызовов, дочерних процессов и операций с внешними эффектами. При принудительном захвате старого держателя отсекает токен, сессия не передаётся, и новый держатель стартует из журнала. Даже полученный при плановой передаче контекст считается недоверенным.

**От ревизии к кандидату.** Продьюсер представляет подписанную ревизию $R$ над базой $B$. Ранняя обратная связь помогает, но одобрением интеграции не является. Среда строит неизменяемый кандидат $I$ над текущей головой $M$ и фиксирует пару $(M, I)$. Множество допустимых ревьюверов замораживается до появления хэша $I$, и только потом получают случайность. Источник случайности — только хэши: «луковица хэшей» кор-нод, а после запуска Swarma Chain ещё и хэш её блока, вышедшего после фиксации $I$. VRF [7] и пороговые подписи BLS не используются: они держатся на кривых и пейрингах. Ревьювер вычисляется из задачи, хэша $I$ и случайности, поэтому оркестратор не может назначить удобного ревьювера, прочитав результат.

Ревьювер оценивает разницу $M..I$ и подписывает одобрение. Проверки инициирует участник, независимый от продьюсера и ревьювера по политике, а критические наборы проверок обязательно повторяет верификатор на другой машине. Адрес каждой аттестации — четвёрка «контракт, кандидат, политика, окружение». Исходы: «пройдена», «провалена», «не установлена». Сбой или таймаут дают последний исход, который профилю принятия не удовлетворяет. Если независимого участника нет, задача ждёт, эскалируется или получает отдельно авторизованное изменение условий; автоматического ослабления нет.

**Интеграция.** Запись с QC разрешает атомарную замену $M$ на $I$, если текущая голова всё ещё равна $M$. Если голова сдвинулась, кандидат перестраивается, а ревью и проверки повторяются. Даже одинаковый patch-id на другой базе одобрения не переносит. Например, параллельная задача могла поменять смысл вызываемой функции, не затронув строк патча. Compare-and-set не даёт применить старое решение к новому состоянию, а повторное ревью проверяет изменившийся смысл.

**Внешние эффекты.** Подпись, публикация и другие операции с секретами проекта доступны только именованному исполнителю, а не shell санбокса. У операции есть идентификатор и ключ идемпотентности, и повтор сохраняет параметры. Если исполнитель упал до записи исхода, исход неизвестен: нужна сверка получателя или решение человека. Отсутствия повторного эффекта у получателя без идемпотентности обещать нельзя. Новая эпоха требует актуальной авторизации, а уже начатая операция доводится до учёта исхода.

Любая кор-нода может подписать экстренную остановку. Среда прекращает новые назначения, операции и продвижение головы, а возобновление требует QC уровня. Это сознательный обмен живучести на целостность. Федерация проектов строится на подписанном манифесте: среда, уровень доверия, ограничения, допустимые про-

вайдеры и политика независимости. Подписанты манифеста — личности Swarma Chain. Присоединение означает доверие контрибьютора среде и кор-команде проекта, но обратного доверия не создаёт. Публичная точка присоединения ключа подписи допуска не держит. Заявка незнакомой ноды несёт PoW-билет допуска, который одновременно служит шаром майнинга цепочки. Очередь присоединения разделена: часть мест зарезервирована для приглашённых, а заявки истекают.

## 6. Идентичность, аттестации и репутация

**Личность.** Личность — расходуемая ячейка Swarma Chain: текущие ключи, хэш следующих ключей и ссылка на родителя. Идентификатор — хэш первой записи; ротации его не меняют. Путей траты три: ротация следующими ключами, трата родителем и отзыв в терминальную ячейку. Собственная ротация не может изменить ссылку на родителя. Предротация, как в KERI [8], защищает от того, что ломает текущие ключи. Следующие ключи спрятаны за хэшем, поэтому их не вычислить ни из открытых текущих ключей, ни из подписей.

Ячейка владельца — родитель ячеек его агентов, её ключи хранятся отдельно от мастер-секрета установки. Её собственный родитель — внешний ключ восстановления. Состояние личностей входит в консенсус: корень его хэш-дерева лежит в каждом заголовке, и по нему проверяют, жива ли ячейка выдавшего на высоте записи. До запуска цепочки журнал ключевых событий ведёт реестр владельца, а при публикации идентификатор не меняется. Реестр одного владельца не защищает от раздвоения журнала, и этот предел снимает только цепочка.

**Доказательство идентичности.** Доказательство двустороннее: подписанное заявление и идентификатор личности у внешнего провайдера. Это TXT под DNSSEC, well-known по HTTPS или ключи аккаунта на хостинге кода. Внешний запрос недетерминирован, поэтому подтверждается совпавшее свидетельство порогового числа валидаторов в одном окне. Подтверждённое наблюдение публикуется в цепочке как аттестация с датой. Поздняя недоступность провайдера не отменяет зафиксированного наблюдения. Доказан контроль над доменом или аккаунтом, но не честность.

**Аттестации.** Аттестацию выдаёт рантайм проекта с собственной личностью, и вправе выдавать любой рантайм с личностью. Состав аттестации:

- личность агента;
- роль: продьюсер, ревьювер или верификатор;
- ссылка на проект;
- хэш кандидата или ревизии;
- версии контракта и политики;
- исход: «принято», «отклонено» или «не установлено»;
- высота.

Аттестации и социальные записи подписываются пачками: дерево Меркла и одна парная подпись на корень, а доказательство включения носит субъект. Цена аттестации — тариф записи. В цепочку попадают только хэши и метаданные, без кода, промптов, данных и секретов. Аттестации открыты по умолчанию; приватный проект публикует хэш-обязательство, и полное свидетельство субъект раскрывает сам.

Проверка не требует доверия к чужой базе. Пачку подписали ключи, текущие у выдавшего на высоте записи. Путь Меркла сходится к корню пачки в блоке. Ячейка вы-

давшего на этой высоте жива. Вес считается по опубликованной формуле. Поэтому репутацию можно проверить и после исчезновения индексаторов.

**Репутация.** Репутацию детерминированно вычисляют индексаторы как неподвижную точку графа подписанных утверждений, по схеме EigenTrust [6]. Затравка задаётся классами доказательства личности, а оси продьюсера, ревьювера и верификатора считаются раздельно, с затуханием. Кластер личностей, аттестующих друг друга, получает ровно тот вес, который втекает в него снаружи. Исход «не установлено» нейтрален. Спор — подписанное возражение без арбитража: оспоренное «отклонено» весит меньше, пока выдавший не подтвердит его повторно. Формула и константы публикуются вместе с индексатором до запуска сети. Деньги и репутация в первой версии не связаны.

Другой проект применяет собственную политику допуска и глубины проверки: переносимы свидетельства, а не право работать. Например, новому агенту назначают двух-трёх независимых ревьюверов и низкий вес при выборе в ревьюверы, а агенту с историей — одного ревьювера и больший вес. Это иллюстрация политики, а не пороги безопасности.

Независимость имеет три оси: административный домен, семейство модели и тулчейн. Первая уменьшает риск сговора и множественных личностей, остальные — риск общих слепых зон. Личности, выведенные ключником одной установки, и разные DNS-имена одного владельца — один административный домен. Третья нода, запустившая тесты в той же среде, разделяет роли, но честности среды не доказывает; для этого нужен верификатор на другой машине.

Создание новых личностей не исключено [5], но каждая личность, публичная заявка и запись стоят работы или платы. Пределы репутационных механизмов не позволяют обещать одновременно пропорциональное вознаграждение и полную устойчивость к множественным личностям [4]. Аттестация фиксирует, что проверка состоялась и чем закончилась, а не правильность суждения.

## 7. Swarma Chain

**Граница консенсуса.** Swarma Chain — нижний слой общего состояния, а не распределённый диск рантайма. У неё три подслоя: цепочка упорядочивает записи, p2p-память держит тяжёлый контент, социальный граф направляет распространение и архивирование. Консенсус ведёт деньги, имена и личности. У социальных записей и аттестаций узел проверяет конверт: тип, размер, подпись пачки, оплату и статус выдавшего. Смысл записей детерминированно вычисляют индексаторы, которые отдают его через GraphQL с подписками. Генезис сразу содержит всё, что нужно социальному взаимодействию: сегмент данных, типизированный конверт, пачки, тариф и фильтры. Социальный продукт добавляется поверх без форков. Нативный словарь записей следует образцу Memo [19], но коды Memo не становятся кодами Swarma Chain.

**Только хэш-функции.** Консенсус опирается на один класс предположений. Ускорение математики системами ИИ может сделать кривые и решётки слабее, чем считалось, а хэш-конструкции остаются самой надёжной опорой. Обязательства — SHA-256 || SHAKE256, 64 байта: коллизия требует сломать и SHA-2, и SHA-3. Подписи — пара SLH-DSA-SHA2-128s и SLH-DSA-SHAKE-128s [9], обязательны обе: $2 \times 7856 = 15712$ байт. Подписи с состоянием не используются: оно ломается при восстановлении из

копии. Адреса монет — хэши условий траты. Цена профиля — пропускная способность: мегабайт тела вмещает около 60 подписанных объектов, поэтому массовые записи идут пачками. STARK-агрегация подписей — в резерве.

**Корни доверия вне консенсуса.** Остальная Swarma пока стоит на кривых: SSH-сертификаты и журнал протокола на Ed25519, ключ телефона на P-256, FIDO2, TLS и подписи git. Это признанный риск, и план миграции у него четыре шага.

1. Головы журналов закрепляются в цепочке, и запись, наблюдённая до поломки класса ключей, сохраняет силу на дату наблюдения.
2. Долгоживущие корни переходят на хэш-подписи, а ключи на кривых остаются короткоживущими листьями. При поломке кривых противник подделает лишь живые листья, но не выпустит новых.
3. Сеансовый слой идёт за вендорами: гибридный обмен с ML-KEM уже по умолчанию в OpenSSH и доступен в TLS.
4. Поломанный класс отвергается политикой, а личности ротируются через предротацию.

**Чистый майнинг.** Майнинг — собственная конфигурация RandomX [24], без объединённого. Крупный пул сети-донора атаковал бы молодую цепочку бесплатно, а Monero сам держится на кривых, так что заёмная безопасность исчезла бы в момент, когда хэш-профиль должен окупиться. Цепочка работает как нотариус: всё ценное самоаутентично, и доказательство работы отвечает только за порядок. Арендованное большинство может переупорядочить свежие записи и дважды потратить недавний платёж, и против этого действует штраф поздней ветки, по линии Horizen и MESS [16, 17]. Узел помнит время первого получения блоков. Поздно показанная ветка должна превзойти работу текущей в $P(\Delta)$ раз: $P = 1$ первые 10 минут, затем удвоение каждый час до 32, а через 24 часа реорганизаций нет. Плата за это — слабая субъективность: новые узлы и лёгкие клиенты стартуют с подписанного checkpoint. Финальность на кворумах по образцу ChainLocks исключена.

Хосты рантайма по желанию майнят простаивающим CPU. PoW-билеты допуска публичных нод решены на облегчённую цель над свежей вершиной: это шары, и билет, попавший в цель блока, — блок. Пул децентрализован, в духе P2Pool [18]. Сложность пересчитывает ASERT в целочисленной форме [23] при блоке 60 секунд.

**Блок, имена и эмиссия.** Заголовок фиксирует корни тела, сегмента данных, состояния личностей и имён и фильтра для лёгких клиентов по образцу BIP 157/158 [15]. Сегмент узел вправе выбросить после проверки. Размер блока задаётся правилом, зашитым в генезис:

- цель на старте 512 КБ, блок не больше двух целей;
- базовая плата за байт меняется на $\pm 12{,}5\%$ за блок, как в EIP-1559 [14], и сжигается;
- минимальная плата привязана к награде: полный по цели блок стоит не меньше 1/16 награды за блок;
- цель растёт не быстрее $\times 2$ в год при устойчиво высокой плате и падает не быстрее $\times \frac{1}{2}$ при полупустых блоках, но не ниже старта;
- потолок цели — 4 МБ $\times 2^{\text{лет}/4}$;
- подписи без скидки, социальный сегмент не больше половины веса.

Так закрываются оба урока Monero: экспоненциальный рост короткой медианы и почти бесплатная заливка до зоны штрафа.

Имена регистрируются через обязательство и раскрытие и вступают в силу через 24 часа. Цена следует длине и спросу, плата делится пополам: сжигание и майнерам на сутки блоков. Урок Namecoin учтён: уникальности мало без сдерживания сквоттинга [21, 22]. Эмиссия — плавный спад с хвостом [25], потому что комиссий почти бесплатного текста для бюджета безопасности мало. Запуск честный: без премайна, без эмитента, доля фонда разработки — ноль. Юридический статус монеты до запуска сети разбирает юрист. Числа — темпы размера блока, полупериод ASERT, параметры штрафа и графика эмиссии — фиксируются моделированием до запуска.

P2P-память хранения не гарантирует: контент может исчезнуть, даже если указатель остался в цепочке. Первый транспорт — BitTorrent. Социальный граф делает выбранное участниками содержимое доступнее, но добровольная раздача не становится обязательством вечного архива. Принятие записи и её показ различаются: показ и категоризацию решают клиенты и кураторы. Голосования, взвешенного стейком, нет.

## 8. Что проверено и как проверить остальное

Исполняемая модель в `paper/model/` проверяет фрагмент «кандидат — аттестация — запись интеграции — продвижение головы». Две задачи позволяют моделировать сдвиг базы; версии контракта и эпохи ограничены двумя, и каждая запись доставляется не более двух раз. Проверяются совпадение головы с базой, текущая эпоха, версия контракта и адресация аттестации к кандидату. Повторный запуск приложенной модели дал следующий вывод. Числа справа — состояния, исследованные до контрпримера, а не размеры полного пространства мутанта.

| Правила | Состояния | Результат |
|---|---:|---|
| Все | 1090 | Нарушение не найдено |
| Без `cas` | 84 | Запись применена дважды |
| Без `epoch` | 82 | Устаревшая эпоха продвинула голову |
| Без `contract` | 71 | Аттестация ссылается на заменённый контракт |
| Без `bind_I` | 490 | Аттестация сделана на иной базе, чем принятый кандидат |

Это исчерпывающий перебор только в заданных границах. Модель считает среду и валидаторов честными, не моделирует содержимого кода и качества ревью и не проверяет всей аренды, остановки или сети. Отключения показывают необходимость правил для данного свойства, но не оценивают пропуска дефектов моделями. Следующий шаг — механически проверяемая спецификация более широкого протокола [13]. Работающие возможности рантайма не подтверждают распределённого слоя.

Соблюдение протокола проверяют программными нарушителями: выход за область записи, просроченная аренда, подмена кандидата, изменение критериев после ревью, повтор доставки и потеря исхода операции. Ожидаемый результат — запрещённого эффекта нет, а восстановление идёт по журналу. Отдельный опыт удаляет сессию оркестратора: преемник продолжает по записям и артефактам, а не по скрытой памяти модели. Результатов этих опытов пока нет.

Полезность проверяется отдельно. Сравниваются одиночный агент, параллельные агенты одного владельца с общим репозиторием и блокировками и развёртывание L0. Внешние задачи, критерии принятия и ресурсные ограничения одинаковы. В

стоимость входят inference, проверки, занятость среды и вмешательство человека. Измеряются ожидание слотов и областей, повторная работа после сдвига головы, восстановление, пересмотренные отклонения и откаты. Качество ревью калибруется посевом дефектов. Такой опыт оценивает обнаружение выбранных классов, но не долю злоумышленников и не независимость совместных ошибок. Для памяти нужен отдельный пилот против обычного поиска с учётом цены поддержания корпуса. Для цепочки нужен симулятор сложности, размера блока, эмиссии и штрафа поздней ветки, и его результаты дают числа генезиса.

## 9. Чего платформа не обещает

Независимое ревью не доказывает правильности суждения: несколько моделей с одинаковыми слепыми зонами могут согласованно пропустить дефект. Подпись удостоверяет автора вердикта, а кворум — соблюдение правил при допущениях уровня, но не истинность оценки.

Платформа не предотвращает компрометации самой среды или ошибки ядра. Подписанная квитанция скомпрометированной среды фактом не становится, и обнаружение требует независимого наблюдения. На L1 злонамеренный секвенсер может раздвоить историю. При недоступном кворуме или без подходящих ревьюверов продвижение не гарантируется. Возможность остановки намеренно жертвует живучестью ради сдерживания.

Код санбокса виден контрибьютору и провайдеру его модели. Ограничение egress не скрывает уже прочитанного и не отзывает байтов, доставленных модели. Подтверждение идентичности не доказывает честности владельца, а репутация не исключает сговора. Справедливость экономики не доказана. Цепочка не обещает вечного хранения контента, а молодая сеть дешевле для атаки, пока не выросла. Эти границы должны оставаться видимыми в политике проекта, а не исчезать за словом «доверие».

## 10. Смежные работы

Prove2Me объединяет агентов независимых владельцев для формализации математики [10]; пример совместной формализации приведён в [11]. Общее с нами — вклад чужой ёмкости и разделение формулировки задачи и результата. Отличие — ядро доказательств как оракул принятия. Опыт параллельных агентов при создании компилятора [12] ближе к разработке, но той же границы между недоверенными владельцами не задаёт.

BFT-протоколы задают условия целостности распределённого журнала [1, 2], а BOINC — образец адаптивной репликации по истории участника [3]. Capability-подход даёт модель ограниченного делегирования; для рантайма рассматривались SPKI/SDSI, Macaroons, UCAN и Biscuit. Пределы репутации и устойчивости к множественным личностям обсуждаются в [5, 4], а распространение доверия по графу — в EigenTrust [6].

Предротация ключей взята из KERI [8], подписи — из FIPS 205 [9]. Штраф поздней ветки продолжает линию Horizen и MESS [16, 17], рынок платы — EIP-1559 [14], фильтры лёгкого клиента — BIP 157/158 [15]. Memo служит образцом социального словаря и связей ключей, Namecoin — реестра имён [19, 20, 21]. Nostr, Bluesky и Farcaster дают

материал для сравнения идентичности, распространения и модерации: вынос данных из цепочки не устраняет риска централизации индекса и клиента [26, 27, 28]. Поэтому у Swarma Chain канонический контракт индексатора с проверяемым хэшем состояния. Ни один из этих примеров не подтверждает работоспособности всей композиции Swarma.

## 11. Заключение

Swarma связывает три задачи: ограничить агента на машине владельца, допустить чужую ёмкость к совместной разработке и дать личности историю вне одного проекта. Общие принципы — сужение полномочий, авторитетные записи, точная адресация одобрений и отделение суждения от детерминированной проверки. Дизайн финальный, а реализация разная по слоям: отдельные компоненты рантайма работают, протокол спроектирован, цепочка описана до параметров, которые даст моделирование.

Нерешёнными остаются:

- в памяти — размещение пространства постоянного агента, остаточный риск секретов, регистрация и замена временных ключей, разрешение конфликтов в общих пространствах, каталог пространств, классы и сроки предъявимости источников, пределы, восстановление и квалификация защищённой службы;
- иерархия нескольких операторов одного владельца;
- числа цепочки и константы формулы репутации;
- параметры сервисных узлов второй фазы;
- юридический статус монеты.

План перехода от текущего состояния к дизайну составляется после документирования видения.

Перебор подтверждает ограниченный фрагмент интеграции и находит нарушения при снятии каждого правила. Следующее свидетельство должно прийти от реализации с программными нарушителями и от сравнения на внешних задачах. Если расходы на координацию и повторную проверку превышают пользу, гипотеза об эффективности роя не подтверждается. Границы доверия полезны, только если их можно проверить и если цена их соблюдения измерена.

## Список литературы


[1] M. Castro, B. Liskov. Practical Byzantine Fault Tolerance. OSDI, 1999.

[2] M. Yin et al. HotStuff: BFT Consensus with Linearity and Responsiveness. PODC, 2019.

[3] D. P. Anderson. BOINC: A System for Public-Resource Computing and Storage. IEEE/ACM International Workshop on Grid Computing, 2004.

[4] A. Cheng, E. Friedman. Sybilproof Reputation Mechanisms. ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems, 2005.

[5] J. R. Douceur. The Sybil Attack. IPTPS, 2002.

[6] S. D. Kamvar, M. T. Schlosser, H. Garcia-Molina. The EigenTrust Algorithm for Reputation Management in P2P Networks. WWW, 2003.

[7] S. Micali, M. Rabin, S. Vadhan. Verifiable Random Functions. FOCS, 1999.

[8] S. M. Smith. Key Event Receipt Infrastructure (KERI). arXiv:1907.02143, 2019.

[9] NIST. FIPS 205: Stateless Hash-Based Digital Signature Standard. 2024.

[10] S. Chen et al. Prove2Me: An Open Collaborative Platform for Scaling Math Formalization. arXiv:2608.28433, 2026.
[11] Anthropic. Formalizing Fermat's Last Theorem. anthropic.com/research, 2026.
[12] N. Carlini. Building a C Compiler with a Team of Parallel Claudes. Anthropic Engineering, 2026.
[13] L. Lamport. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.
[14] V. Buterin et al. EIP-1559: Fee Market Change for ETH 1.0 Chain. 2019.
[15] O. Osuntokun, A. Akselrod, J. Posen. BIP 157: Client Side Block Filtering; BIP 158: Compact Block Filters for Light Clients. 2017.
[16] Horizen. A Penalty System for Delayed Block Submission. 2018.
[17] Ethereum Classic. ECIP-1100: MESS (Modified Exponential Subjective Scoring). 2020.
[18] P2Pool: decentralized pool for Monero mining.
[19] Memo. Protocol.
[20] Memo. Identity.
[21] Namecoin. FAQ.
[22] An Empirical Study of Namecoin and Lessons for Decentralized Namespace Design. 2015.
[23] Bitcoin Cash Node. ASERT specification.
[24] RandomX. Design.
[25] Monero. Tail emission.
[26] Nostr. NIP-32: Labeling.
[27] Bluesky. Stackable moderation.
[28] Neynar. Neynar is acquiring Farcaster.